

and after `init` (we'll discuss `__init__` in “Methods, `__init__()`, and `self`” later in this chapter):

```
❶ class WizCoin:
❷     def __init__(self, galleons, sickles, knuts):
        """Create a new WizCoin object with galleons, sickles, and knuts."""
        self.galleons = galleons
        self.sickles   = sickles
        self.knuts     = knuts
        # NOTE: __init__() methods NEVER have a return statement.

❸     def value(self):
        """The value (in knuts) of all the coins in this WizCoin object."""
        return (self.galleons * 17 * 29) + (self.sickles * 29) + (self.knuts)

❹     def weightInGrams(self):
        """Returns the weight of the coins in grams."""
        return (self.galleons * 31.103) + (self.sickles * 11.34) + (self.knuts *
5.0)
```

This program defines a new class called `WizCoin` using a `class` statement ❶. Creating a class creates a new type of object. Using a `class` statement to define a class is similar to `def` statements that define new functions. Inside the block of code following the `class` statement are the definitions for three methods: `__init__()` (short for *initializer*) ❷, `value()` ❸, and `weightInGrams()` ❹. Note that all methods have a first parameter named `self`, which we'll explore in the next section.

As a convention, module names (like `wizcoin` in our `wizcoin.py` file) are lowercase, whereas class names (like `WizCoin`) begin with an uppercase letter. Unfortunately, some classes in the Python Standard Library, such as `date`, don't follow this convention.

To practice creating new objects of the `WizCoin` class, enter the following source code in a separate file editor window and save the file as `wcexample1.py` in the same folder as `wizcoin.py`:

```
import wizcoin

❶ purse = wizcoin.WizCoin(2, 5, 99) # The ints are passed to __init__().
print(purse)
print('G:', purse.galleons, 'S:', purse.sickles, 'K:', purse.knuts)
```